

06

1시간

기업 보안

마스킹 Hook

접근 범위

승인 정책

조직 설정

감사 로그

‘기업 보안’ 주제를 다루는 이유

Claude Code는 파일을 읽고 셸 명령을 실행합니다. 업무에 도입하면 **사내 코드와 고객 데이터가 Claude Code를 통과합니다.**

보안 검토는 세 가지를 묻습니다.

- 사내 데이터가 어디까지 나가는가
- 실행할 수 있는 명령을 무엇으로 제한하는가
- 사고가 났을 때 무엇을 근거로 확인하는가

챕터 6에서는 데이터 보호와 권한 제어, 감사 로깅을 Claude Code의 설정과 Hook으로 구현합니다.

이 챕터에서 다루는 내용

주제	내용	실습
유출 지점	데이터가 유출되는 경로와 설정이 놓이는 층	1
민감정보 마스킹	도구가 돌려준 결과를 모델이 보기 전에 가리는 Hook	2
외부 전송 차단	네트워크로 나가는 도구 인자를 실행 전에 검사	3
환경변수 외부화 · 파일 제외	자격증명을 두는 자리와 저장소에서 빼는 대상	4
접근 범위 제한	읽기 금지 경로 지정	4
승인 정책 · 화이트리스트	물어볼 도구와 바로 실행할 명령의 구분	5
감사 로깅	도구 호출 기록과 Telemetry, SIEM 연동	6

이 챕터에서는 앱 코드를 고치지 않습니다.
 바꾸는 것은 `.claude/` 의 설정과 Hook뿐입니다.

데이터가 유출되는 네 경로

경로	지나가는 것	이 챕터의 장치
화면	도구가 돌려준 파일 내용	마스킹 Hook
모델 API	같은 내용이 요청에 실려 전송	마스킹 Hook
세션 기록 파일	주고받은 내용 전부가 홈 폴더에 쌓임	마스킹 Hook
MCP 서버	연결한 서버로 넘어가는 도구 인자	권한 규칙

- **도구 뒤에서 가리는 세 경로:** 화면과 모델 요청, 세션 기록에는 같은 값이 실립니다. 결과가 모델에 전달되기 전에 한 번 가리면 셋이 함께 가려집니다.
- **도구 앞에서 막아야 하는 경로:** MCP 서버로는 도구 인자가 그대로 나갑니다. 호출된 뒤에는 늦으므로 권한 규칙으로 막습니다.

한 번 나간 값은 되돌릴 수 없습니다.

세션 기록은 지울 수 있지만 모델 요청과 MCP 서버로 나간 것은 회수하지 못합니다.

설정의 적용 계층

	개인	프로젝트	프로젝트 로컬	조직
우선순위	4	3	2	1
지정하는 곳	~/.claude/ settings.json	.claude/ settings.json	.claude/ settings.local.json	managed- settings.json
적용 대상	본인의 모든 프로젝트	저장소 공유자 모두	이 프로젝트에서 본인만	설정을 배포한 PC의 모든 작업
고치는 사람	본인	저장소 공유자	본인	시스템 관리자

같은 설정이 여러 층에 있으면 조직 층의 값이 적용됩니다.

우선순위는 조직, 프로젝트 로컬, 프로젝트, 개인 순으로 낮아지고, 높은 층의 값이 낮은 층의 값을 덮어씹습니다. 다만 권한 규칙은 덮어쓰이지 않고 층마다 합쳐집니다. 예를 들어 프로젝트 로컬에 `Bash(rm:*)` 를 `allow` 로 적어도, 우선순위가 낮은 프로젝트 설정의 `deny` 에 같은 명령이 있으면 `rm` 은 차단됩니다.

실습 1 · 유출 지점 확인

- 1 요청: `samples/개발2팀.csv` 의 첫 두 줄과 `EXP-2026-0230` 이 있는 줄을 셀 명령으로 보여 줘
- 2 화면에 카드번호와 계좌번호, 주민등록번호가 그대로 나오는지 확인
- 3 요청: 내 홈 폴더의 `.claude/settings.json` 을 읽어 줘
- 4 승인한 뒤 화면에 API 키가 그대로 나오는지 확인
- 5 요청: `~/.claude/projects` 아래 세션 기록 파일(`.jsonl`)에서 카드번호와 API 키를 찾아 줘

Claude Code가 찾아낸 값 중 API 키. 4번에서 확인한 값과 동일

```
sk-ant-api03-XXXXXXXXXXXX
```

이미 나간 값은 설정을 바꿔도 지울 수 없습니다.

승인 한 번으로 API 키가 화면에 표시됐고, 같은 값이 모델 요청과 세션 기록에도 남았습니다.

이 실습의 API 키는 교육용입니다. 세션 기록에 평문으로 남으므로 강의가 끝나면 키를 폐기해 주세요.

Hook이 개입하는 두 시점

	PreToolUse	PostToolUse
시점	도구가 실행되기 전	도구가 실행된 뒤, 모델이 보기 전
다루는 것	도구에 넘기는 인자	도구가 돌려준 결과
돌려주는 값	updatedInput permissionDecision	updatedToolOutput
막는 것	네트워크 외부로 전송되는 데이터	모델과 세션 기록에 남는 결과

데이터가 네트워크 외부로 전송되는 도구는 **PreToolUse** 에서만 막을 수 있습니다.

MCP 서버 호출이나 웹 요청은 도구가 실행되는 순간 이미 전송됩니다. **PostToolUse** 는 그 뒤에 모델이 보는 것만 바꿉니다.

Hook이 주고받는 값

실제로 오간 값

```
# PreToolUse · 들어오는 것
{ "tool_name": "WebFetch",
  "tool_input": { "url": "...?id=4667-2280-5514-9073" }}

# PreToolUse · 돌려주는 것
{ "hookSpecificOutput": {
  "hookEventName": "PreToolUse",
  "permissionDecision": "deny",
  "permissionDecisionReason": "..." }}

# PostToolUse · 들어오는 것
{ "tool_name": "Read", "tool_response": { "type": "text",
  "file": { "filePath", "content", "numLines",
    "startLine", "totalLines" }}}

# PostToolUse · 돌려주는 것
{ "hookSpecificOutput": { "hookEventName": "PostToolUse",
  "updatedToolOutput": { content 만 고쳐 그대로 }}}}
```

- **PreToolUse** 가 받는 것: **tool_input** 입니다. 이 값을 바꾸거나(**updatedInput**) 호출을 거부합니다 (**permissionDecision**).
- **PostToolUse** 가 받는 것: **tool_response** 입니다. 고쳐서 **updatedToolOutput** 으로 돌려줍니다. 받은 모양을 그대로 지켜야 합니다. 어긋나면 오류가 기록되고 원본이 가려지지 않은 채 전달됩니다.
- 한 **matcher**에 하나만: 출력을 다시 쓰는 Hook을 여럿 걸면 병렬로 실행되어 어느 것이 적용될지 정해지지 않습니다.
- 가려지지 않는 것: 결과는 가려도 **tool_input** 은 세션 기록에 남습니다.

실습 2 · 결과 마스킹

요청 · 한 번에 전달

카드번호와 계좌번호, 주민등록번호가
모델에 전달되지 않게 하는 Hook을 만들어 줘.

1. 스크립트 파일 경로 및 이름은
`${CLAUDE_PROJECT_DIR}/.claude/hooks/mask.py`
2. PostToolUse 로 걸고 대상은 Bash 와 Read
3. 도구가 돌려준 결과에서 세 패턴을 가려
`updatedToolOutput` 으로 돌려줌
4. 받은 응답의 모양을 그대로 지키고 값만 고침
5. `.claude/settings.json` 에 등록

- 1 `.claude/` 아래에 파일을 쓰겠다는 승인 프롬프트가 나타나면 허용
- 2 `/hooks` 로 `PostToolUse` 에 `mask.py` 가 등록됐는지 확인
- 3 실습 1의 첫 요청을 다시 전송해 세 가지 값이 모두 가려졌는지 확인
- 4 **요청:** 셸 명령을 쓰지 말고 Read 도구로 `samples/개발2팀.csv` 를 읽어 줘
- 5 같은 값이 가려졌는지 확인

`.claude/` 는 보호 경로입니다.

편집을 자동 수락하는 모드에서도 승인을 묻습니다.

실습 2 실행 결과

Hook을 등록하기 전

```
EXP-2026-0225,E21349,오세훈,...,4667-2280-5514-9073,하나 311-890337-20108,950127-2661043
```

Hook을 등록한 뒤 같은 요청

```
EXP-2026-0225,E21349,오세훈,...,****-****-****-9073,하나 ***-*****-**108,950127-*****
```

- **한 Hook을 두 도구에 함께 거는 이유:** 가리는 대상이 명령이 아니라 결과입니다. `Bash` 로 열든 `Read` 로 열든 돌아오는 값의 자리는 같습니다.

명령을 바꾸지 않으므로 승인 절차가 그대로입니다.
여러 명령을 이어 붙여도 합쳐진 출력 전체가 가려집니다.

네트워크 외부로 전송되는 도구 호출

앞의 Hook은 돌아온 결과를 가립니다. 도구가 값을 네트워크 외부로 전송하면 가리는 시점이 이미 늦습니다.

도구	실행되는 순간 일어나는 일
WebFetch · WebSearch	인자가 외부 서비스로 전송
MCP 도구	인자가 연결한 서버로 전송
Bash 의 curl 류	셸이 직접 전송

이미 전송된 데이터는 회수하지 못합니다.

PostToolUse 는 모델이 보는 것만 바꿉니다. 앞에서 「도구 앞에서 막아야 하는 경로」라고 한 것이 이 호출들입니다.

실습 3 · 네트워크 외부 전송 차단

요청 · 한 번에 전달

데이터가 네트워크 외부로 전송되는
도구 호출을 검사하는 Hook을 만들어 줘.

1. 스크립트 파일 경로 및 이름은
`${CLAUDE_PROJECT_DIR}/.claude/hooks/guard.py`
2. PreToolUse 로 걸고 대상은
WebFetch, WebSearch, MCP 도구
3. 도구 인자에 카드번호나 계좌번호,
주민등록번호가 있으면 `permissionDecision` 을
`deny` 로 돌려주고 이유를 적음
4. MCP 는 `mcp__서버명__.*` 형태로 적어야 걸림
5. `.claude/settings.json` 에 등록

- 1 `/hooks` 로 `PreToolUse` 에 `guard.py` 가 등록됐는지 확인
- 2 요청: 다음 URL을 WebFetch로 가져와 줘:
`https://example.com/lookup?id=4667-2280-5514-9073`
- 3 거부 사유가 표시되는지 확인
- 4 요청: FastAPI 의 BackgroundTasks 사용법을 웹에서 찾아 줘
- 5 그대로 진행되는지 확인

네트워크가 막혀 있어도 이 실습은 진행됩니다.

거부는 도구가 실행되기 전에 일어나므로 외부 접속이 필요 없습니다.

실습 3 실행 결과

인자에 카드번호가 실린 호출

> 다음 URL을 WebFetch로 가져와 줘: `https://example.com/lookup?id=4667-2280-5514-9073`

WebFetch 차단됨

외부 요청 인자에 카드번호로 보이는 문자열이 있어 차단했습니다

민감정보가 없는 호출

> FastAPI 의 BackgroundTasks 사용법을 웹에서 찾아 줘
WebSearch 진행

모델의 판단에 기대지 않습니다.

모델이 값을 빼고 검색어를 만들기도 하지만 매번 그렇다는 보장이 없습니다.

자격증명을 두는 자리

대상	두는 자리
API 키	운영체제 환경변수. 이 과정에서는 <code>~/.claude/settings.json</code> 의 <code>env</code> 에 두므로 그 파일을 <code>deny</code> 로 차단
공용 환경변수	<code>.claude/settings.json</code> 의 <code>env</code> . 자격증명이 아닌 값만
앱이 읽는 비밀값	<code>.env</code> 파일. <code>.gitignore</code> 와 <code>permissions.deny</code> 에 함께 등록
발급받아 쓰는 키	<code>apiKeyHelper</code> . 키를 적어 두지 않고 명령을 실행해 그때그때 발급

자격증명을 프로젝트 설정에 두지 않습니다.

`.claude/settings.json` 은 저장소에 올라가 공유자 모두에게 전달됩니다. 개인 키는 사용자 설정에 두고, 앱이 읽는 값은 `.env` 로 분리합니다.

권한 설정의 구성

항목	정하는 것
deny	거부할 대상. 승인으로 넘길 수 없음
ask	기본이 허용이어도 물어볼 대상
allow	문지 않고 실행할 대상
defaultMode	어느 규칙에도 걸리지 않았을 때의 기본 동작
additionalDirectories	작업 디렉터리 밖에서 추가로 허용할 경로
enabledMcpjsonServers	<code>.mcp.json</code> 의 서버 중 연결할 서버만 지정

deny · ask · allow 는 적힌 차례대로 평가됩니다.

deny 에 먼저 걸리면 allow 에 더 구체적으로 적혀 있어도 거부됩니다. 표기는 Bash(`uv run pytest:*`) 처럼 도구(대상) 형태로 쓰고, MCP 도구는 `mcp__서버명__도구명` 으로 적습니다.

실습 4 · 접근 범위 제한

- 1 차단 요청: `permissions.deny` 에 `Read(~/.ssh/**)`, `Read(~/.aws/**)`, `Read(**/.env*)`, `Read(**/secrets/**)`, `Read(**/credentials/**)` 를 넣어 줘
- 2 보존 기간 요청: 세션 기록이 오래 남지 않게 `cleanupPeriodDays` 를 14 로 넣어 줘
- 3 요청: `.claude/settings.json` 의 `env` 블록에 자격증명이 들어 있지 않은지 확인해 줘
- 4 추가 차단 요청: 이 과정에서는 API 키를 `~/.claude/settings.json` 에 뒀. `Read(~/.claude/settings.json)` 도 `deny` 에 넣어 줘
- 5 요청: 내 홈 폴더의 `.claude/settings.json` 을 읽어 줘
- 6 권한 규칙이 돌려주는 문구를 확인

규칙이 막는 것은 Agent의 도구 호출뿐입니다.

`uv run settle` 은 앱이 파일을 직접 읽으므로 걸리지 않고, `/rewind` 나 세션 이어가기도 Claude Code의 내부 동작이라 영향이 없습니다.

실습 4 실행 결과

.claude/settings.json · 실습 4를 마친 뒤

```
{ "permissions": {
  "deny": [ "Read(~/.ssh/**)", "Read(~/.aws/**)", "Read(**/.env*)",
    "Read(**/secrets/**)", "Read(**/credentials/**)",
    "Read(~/.claude/settings.json)" ] },
  "cleanupPeriodDays": 14 }
```

권한 규칙이 홈 폴더 `.claude` 읽기를 막았을 때 나오는 문구

```
File is in a directory that is denied by your permission settings.
```

Hook은 값을 가리고, 권한 규칙은 접근 자체를 막습니다.

업무에 필요한 파일은 가린 채로 계속 다룰 수 있어야 하므로 Hook으로 처리하고, 볼 이유가 없는 대상은 `permissions.deny` 에 적습니다.

새 세션의 기본 모드 고정

.claude/settings.json · 모드를 파일로 정하는 두 항목

```

"permissions": { "defaultMode": "manual", "disableBypassPermissionsMode": "disable" }

```

defaultMode 값	새 세션이 하는 일
manual	읽기만 묻지 않고 실행. 나머지는 매번 확인
acceptEdits	파일 편집과 mkdir · mv 같은 명령까지 묻지 않음
plan	읽고 계획만 세움. 승인 전에는 편집하지 않음
auto	전부 실행한 뒤 안전성 검사. 개인 설정에만 적용됨
bypassPermissions	전부 묻지 않고 실행. disableBypassPermissionsMode 로 진입을 막음

챕터 3의 Shift+Tab 은 그 세션에만, defaultMode 는 새 세션마다 적용됩니다.
 모드는 묻는 범위를 정할 뿐이고, deny 규칙은 bypassPermissions 를 포함한 모든 모드에서 그대로 평가됩니다.

실습 5 · 승인 정책

- 1 **허용 요청:** 승인 정책을 정리해 줘. `defaultMode` 를 `manual` 로 두고, 되풀이하는 `git status` 와 `git diff`, `git log` 는 `allow` 에 넣어 줘. 규칙은 `Bash(git status:*)` 처럼 콜론과 별표를 붙인 형태로 써 줘
- 2 **제한 요청:** `git commit` 은 매번 묻도록 `ask` 에 넣고, 되돌릴 수 없는 `rm` 과 `git push` 는 `deny` 에 넣어 줘. 규칙은 앞과 같이 콜론과 별표를 붙인 형태로 써 줘
- 3 신뢰 대화가 나타나면 목록을 읽고 수락. 나타나지 않으면 Claude Code를 종료하고 다시 실행
- 4 **허용 확인 요청:** `git status` 로 지금 변경 상태를 보여 줘
- 5 승인을 묻지 않고 바로 실행되는지 확인
- 6 **차단 확인 요청:** `git push` 로 원격에 올려 줘
- 7 거부되는지 확인

프로젝트 설정의 **allow** 는 신뢰 대화를 수락해야 적용됩니다.

규칙을 추가하면 대화가 다시 나타나며 새로 넣은 항목을 보여 줍니다. 수락하지 않으면 그 규칙만 무시되고 나머지 설정은 그대로 동작합니다.

실습 5 실행 결과

신뢰 대화를 수락하기 전에 나오는 알림

```
Ignoring 3 permissions.allow entries from .claude/settings.json:
this workspace has not been trusted.
```

신뢰 대화를 수락한 뒤

```
# git status → 승인 프롬프트 없이 실행
On branch main / Untracked files: a.txt
# git push origin main → 거부
Permission to use Bash with command git push origin main has been denied.
```

allow 는 신뢰를 수락해야, **deny** 는 곧바로 적용됩니다.
수락 전에는 **allow** 만 무시되고 **deny** 와 **ask** 는 그대로 동작합니다.

조직이 강제하는 설정

MDM이나 그룹 정책으로 배포 · C:\Program Files\ClaudeCode\managed-settings.json

```
{
  "permissions": {
    "defaultMode": "manual", "deny": ["Read(.env)", "Read(~/.ssh/**)"],
    "allowManagedPermissionRulesOnly": true
  },
  "enabledMcpjsonServers": ["context7"],
  "forceLoginOrgUUID": "조직 UUID"
}
```

- **규칙 병합 차단:** `allowManagedPermissionRulesOnly` 를 켜면 조직이 적은 규칙만 남고 아래 층의 규칙은 합쳐지지 않습니다.
- **연결 제한:** 목록에 적은 MCP 서버만 연결됩니다. 목록 밖 서버는 `.mcp.json` 에 있어도 시작되지 않습니다.
- **로그인 제한:** `forceLoginOrgUUID` 를 지정하면 다른 조직 계정으로 로그인하거나 환경변수에 넣은 API 키로 접속하는 경로가 차단됩니다.

사용 기록과 수집

	세션 기록 파일	Hook 감사 로그	OpenTelemetry
남기는 주체	Claude Code 기본	직접 만든 Hook	Claude Code 기본
담기는 것	주고받은 내용 전부	정한 항목만	이벤트와 속성
남는 자리	홈 폴더	정하는 대로	수집 서버
켜는 방법	항상 켜져 있음	Hook 등록	환경변수

조직이 모아 보려면 Hook이나 Telemetry로 내보내야 합니다.

세션 기록은 개인 PC의 홈 폴더에만 쌓이고, 실습 4에서 정한 보존 기간이 지나면 지워집니다. 실습 6에서 두 방법을 모두 확인합니다.

실습 6-1 · Hook 감사 로그

- 1 **요청:** 모든 도구 호출을 `.claude/audit.jsonl` 에 남기는 `PostToolUse` Hook을 `${CLAUDE_PROJECT_DIR}/.claude/hooks/audit.py` 로 만들어 줘. 시각과 도구 이름, 인자 요약의 한 줄씩 적고, 민감정보는 남기지 마. `.claude/settings.json` 에 등록하고 `.claude/audit.jsonl` 은 `.gitignore` 에 넣어 줘
- 2 `/hooks` 로 `PostToolUse` 에 `audit.py` 가 등록됐는지 확인. 보이지 않으면 Claude Code를 종료하고 다시 실행
- 3 **요청:** `uv run pytest` 를 실행해 줘
- 4 **요청:** `samples` 폴더에서 교육훈련비가 있는 줄을 셀 명령으로 찾아 줘
- 5 **요청:** `docs/경비규정.md` 를 `Read` 도구로 읽어 줘
- 6 `.claude/audit.jsonl` 을 열어 세 호출이 한 줄씩 남았는지 확인

남길 항목을 직접 고를 수 있는 것이 Hook 감사 로그의 장점입니다.

세션 기록은 주고받은 내용 전부가 그대로 쌓이지만, 여기서는 시각과 도구 이름, 인자 요약만 적습니다.

실습 6-1 실행 결과

.claude/audit.jsonl · Hook이 남긴 기록

```
{"time": "2026-08-18T23:38:40+09:00", "tool": "Bash", "args": "uv run pytest -q"}
{"time": "2026-08-18T23:38:44+09:00", "tool": "Bash", "args": "grep 교육훈련비 samples/*.csv"}
{"time": "2026-08-18T23:38:51+09:00", "tool": "Read", "args": "docs/경비규정.md"}
```

- **인자 요약만 남는 이유:** 도구가 돌려준 내용은 적지 않습니다. 감사에 필요한 것은 무엇을 언제 호출했는가이고, 내용까지 남기면 이 파일이 새로운 유출 지점이 됩니다.
- **저장소에 올리지 않는 이유:** 개인의 작업 이력이므로 `.gitignore` 에 넣습니다. 조직이 모아 보는 것은 다음 실습의 Telemetry로 합니다.

Hook이 실패해도 도구 실행은 막지 않습니다.

감사 기록이 목적이므로 `audit.py` 는 종료 코드로 차단하지 않습니다.

실습 6-2 · Telemetry

- 1 새 PowerShell 창에서 아래 두 줄을 입력 (macOS와 Linux는 `export 이름=값` 으로 설정)

```
$env:CLAUDE_CODE_ENABLE_TELEMETRY = "1"
$env:OTEL_LOGS_EXPORTER = "console"
```

- 2 같은 창에서 `claude -p "telemetry.txt 를 .gitignore 에 넣어 줘" --permission-mode acceptEdits > telemetry.txt` 실행
- 3 `telemetry.txt` 를 열어 `claude_code.tool_decision` 이벤트 확인
- 4 같은 파일에서 `user_prompt` 와 `tool_result`, `api_request` 도 찾아보기
- 5 `.gitignore` 에 `telemetry.txt` 가 들어갔는지 확인

대화형 화면에서는 이벤트가 보이지 않습니다.

`console` exporter는 표준 출력에 쓰지만 대화형 Claude Code가 화면을 다시 그리면서 덮어쓰기 때문입니다. 파일로 받아 열어야 확인할 수 있습니다.

실습 6-2 실행 결과

telemetry.txt · `tool_decision` 이벤트. 개인 식별자는 가려서 표기

```
body: "claude_code.tool_decision"
attributes: {
  "session.id": "43f11918-...", "organization.id": "f71e38d0-...",
  "user.email": "...", decision: "accept", source: "config", tool_name: "Edit" }
```

한 번의 요청에서 나온 이벤트

```
user_prompt 1 · tool_decision 2 · tool_result 2
api_request 4 · assistant_response 2 · mcp_server_connection 8
```

모든 이벤트에 계정과 조직 식별자가 함께 담겨 전송됩니다.

요청 한 번에 이벤트가 열 건 넘게 나가므로 무엇을 내보낼지 먼저 정합니다.

SIEM 연동

PowerShell · 중앙 수집기로 내보내는 환경변수

```
# 1. Telemetry 활성화
$env:CLAUDE_CODE_ENABLE_TELEMETRY = "1"

# 2. 내보내기 선택 (필요한 것만)
$env:OTEL_METRICS_EXPORTER = "otlp" # otlp, prometheus, console, none
$env:OTEL_LOGS_EXPORTER = "otlp" # otlp, console, none

# 3. OTLP 엔드포인트
$env:OTEL_EXPORTER_OTLP_PROTOCOL = "grpc"
$env:OTEL_EXPORTER_OTLP_ENDPOINT = "http://localhost:4317"

# 4. 인증 (필요한 경우)
$env:OTEL_EXPORTER_OTLP_HEADERS = "Authorization=Bearer your-token"

# 5. 내보내기 간격 (기본 60000 · 5000)
$env:OTEL_METRIC_EXPORT_INTERVAL = "10000"
$env:OTEL_LOGS_EXPORT_INTERVAL = "5000"
```

- **SIEM(Security Information and Event Management):** 여러 시스템의 로그를 한곳에 모아 이상 징후를 찾는 보안 관제 도구입니다. Splunk와 Elastic, Sentinel 같은 제품이 있습니다.
- **지표와 이벤트의 분리:** 내보낼 신호를 각각 정합니다. 필요한 쪽만 구성하면 됩니다.
- **본문 처리:** 프롬프트와 도구 인자의 본문은 기본으로 `<REDACTED>` 로 가려지고 글자 수만 남습니다.

실습 6-2에서 `console` 로 본 이벤트가 그대로 수집기로 전송됩니다.

항목별 설정과 이벤트 목록은 [공식 문서](#)에 있습니다.

수집한 기록의 활용

하는 일	쓰는 이벤트와 지표
부서별 비용 집계	<code>cost.usage</code> · <code>token.usage</code> 를 <code>OTEL_RESOURCE_ATTRIBUTES</code> 의 부서 속성으로 분리
승인 정책 점검	<code>tool_decision</code> 의 <code>decision</code> 과 <code>source</code> 로 거부된 호출과 차단 주체 확인
도구 실패 추적	<code>tool_result</code> 의 <code>success</code> 와 <code>error_type</code> , <code>duration_ms</code> 집계
연결 서버 감사	<code>mcp_server_connection</code> 의 <code>server_scope</code> 와 <code>status</code> 확인
요청 단위 흐름 재구성	<code>prompt.id</code> 로 한 요청에서 나온 이벤트를 하나로 결합

흩어진 이벤트를 `prompt.id` 로 이어 볼 수 있습니다.

요청 하나가 이벤트 여러 건으로 나뉘어 전송되지만, 모든 이벤트에 같은 `prompt.id` 가 별도 설정 없이 붙습니다.

확인 항목

- ☐ 세션 기록에 남은 카드번호와 API 키 확인
- ☐ `.claude/hooks/mask.py` 생성
- ☐ `.claude/hooks/guard.py` 생성
- ☐ 민감정보 없는 웹 요청은 통과 확인
- ☐ `cleanupPeriodDays` 로 세션 기록 보존 기간 단축
- ☐ `defaultMode` 를 `manual` 로 변경
- ☐ 신뢰 대화 수락 후 `allow` 적용 확인
- ☐ `.claude/hooks/audit.py` 생성과 `.gitignore` 추가
- ☐ `telemetry.txt` 에서 `tool_decision` 이벤트 확인
- ☐ `~/.claude/settings.json` 읽기 승인 후 API 키 노출 확인
- ☐ `Bash` 와 `Read` 결과에서 세 가지 값 마스킹 확인
- ☐ 카드번호가 담긴 `WebFetch` 호출 차단 확인
- ☐ `deny` 에 `.ssh` · `.aws` · `.env` · `secrets` · `credentials` 추가
- ☐ `env` 블록에 자격증명이 있는지 확인
- ☐ `allow` 와 `ask` 에 명령 등록
- ☐ `rm` 차단 확인
- ☐ `audit.jsonl` 에 세 호출이 한 줄씩 남았는지 확인
- ☐ `uv run pytest` 에서 `19 passed` 유지